PacaPet, a Proposal-Governance Architecture for Persistent Local LLM Agents

Erina Karati
University of Minnesota
erina.karati@gmail.com

Daniel Kluver
University of Minnesota
kluve018@umn.edu

Abstract

Persistent embodied agents driven by local LLMs fail differently from chatbots, because model errors immediately contradict visible body state, such as eating from an empty bowl or sleeping while energetic. We report on PacaPet, an early-phase desktop virtual pet whose behavior is partly proposed by a small local LLM. Rather than evaluate the pet as a whole product, we focus on one architectural choice: separating proposal from governance. On each 30-second decision cycle (a *heartbeat*), a 3B Llama 3.2 proposes a short JSON plan, while deterministic code owns body state, action effects, feasibility checks, fallback behavior, and user-taught cues. Three inspectable gates, for action preconditions, safety policy, and cue compatibility, filter proposals before any action mutates the world. In scaffold-level tests spanning 27 workflow scenarios, 84 feasibility cases, three gate ablations, a 64-decision live Ollama trace, and a 120-decision stress suite, malformed or invalid proposals degrade into governed behavior with no policy-blocked action executed, no impossible resource consumption, and no out-of-bounds stat mutation. The model still supplies visible variation, short-horizon intent, and expression. We do not measure felt coherence, attachment, or engagement; those require user studies beyond this report.

1 Introduction

Emerging dialogue systems are increasingly agentic and can remember, plan, coordinate with users, and act across contexts. Persistent agents add a failure mode that ordinary chatbots can often hide. If a model proposes an action that contradicts visible state, the world immediately becomes incoherent. A local companion cannot say it is eating from an empty bowl, sleeping when its body is energetic, or obeying a learned cue that conflicts with a critical drive.

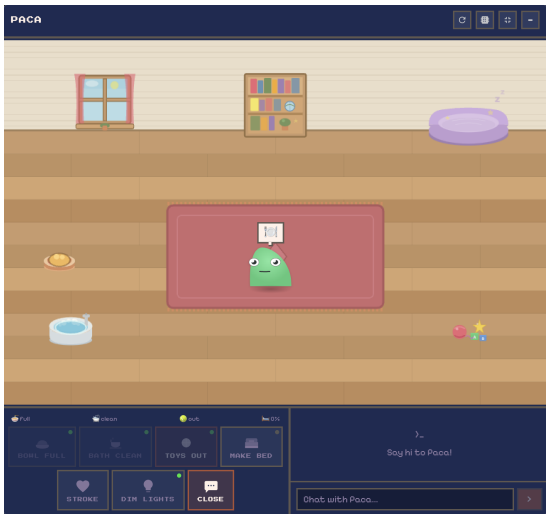


Figure 1: PacaPet's desktop surface. Users change the room through controls and short cues; the pet responds through its heartbeat loop.

PacaPet explores this problem in a small but demanding setting. It is an always-on desktop virtual pet whose behavior is partly proposed by a local language model (LLM). The user does not press a "feed" command that directly changes a stat. Instead, they fill a bowl, draw a bath, deploy toys, drag the pet toward a zone, or type a short cue. The pet then decides what to do on the next *heartbeat*, a decision cycle that fires roughly every 30 seconds.

The PacaPet world. Figure 1 shows the always-on-top widget. A small room contains four affordances: a food bowl, a bathtub, a bed, and a toy zone. The user can fill the bowl, draw a bath, deploy toys, tidy the bed, dim the lights, drag the pet toward any zone, and type short cues into a chat panel. The pet carries continuous body state (hunger, energy, cleanliness, boredom, mood) and transient conditions such as nausea after overeating or feeling cold

10.21437/WoChat.2026-2

when the lights are dim. Every heartbeat, the pet observes the room and its body, then picks its next action; the user sees the outcome as animation, movement between zones, or an expressive change on the pet's face. There is no command line, and every input is embodied.

PacaPet treats the LLM as a proposal source and treats deterministic software as the governor of visible state. This setting is intentionally modest, but the architectural pressure is general. A small model can add variation, short-horizon intent, and expressive language, yet cannot be trusted to own the body.

The language contribution is deliberately narrow. Short utterances become grounded cues for future embodied behavior rather than open-ended companion dialogue or a separate assistant transcript. This makes PacaPet a test case for situated cue learning where user phrases, visible state, autonomous action, and persistent memory must remain consistent over time.

Contribution. PacaPet is in an early exploratory phase. We do not evaluate it as a whole product, and we do not compare it against an alternative architecture. Instead, we report on the architectural choices that let a small local model drive a persistent embodied pet without owning the body. We describe (1) a *proposal-governance* separation in which the model proposes and deterministic code governs, (2) a *heartbeat* loop in which JSON action sequences are filtered by three inspectable gates covering action preconditions, safety policy, and cue compatibility, with a deterministic fallback whenever the model output is unusable, and (3) an inspectable *cue-action memory* that lets short user phrases become future behavioral hints only when repeated situated feedback supports them, and only when contextually compatible. Our evaluation asks whether this scaffold intercepts the invalid proposals it was designed to catch; it does not attempt to measure user attachment or enjoyment.

2 Related Work

PacaPet sits between agentic dialogue, believable agents, and grounded action. *Generative Agents* simulate small communities of language-driven characters that reflect on their day and reuse those reflections to plan later actions (Park et al., 2023), and *Reflexion* similarly lets an agent store verbal feedback from failed attempts and consult it on retry (Shinn et al., 2023). Both show that language agents can accumulate experience without weight updates. *SayCan* grounds a language plan in a robot's currently-executable skills by weighting each candidate action by both its language likelihood and a learned affordance score (Ahn et al., 2022), and *ReAct* interleaves the model's reasoning traces with actual tool or environment calls so that the next thought is conditioned on real observations (Yao et al., 2023). Both argue that LLM proposals need grounding in what the world can support. Believable-agent work also argues that emotion and visible behavior are part of the agent interface rather than cosmetic output (Bates, 1994).

PacaPet applies these ideas at a smaller operating point with a 3B local model, a consumer desktop, a heartbeat loop, and visible embodied state that cannot pause while the model deliberates. Its local-first constraint changes the design pressure, because private household interaction should not require remote inference or opaque server-side memory.

Voyager, an embodied LLM agent in Minecraft, externalizes learned skills as reusable code stored in a growing skill library (Wang et al., 2023). PacaPet differs by treating reuse as governed cue-conditioned behavior in a persistent local companion rather than benchmark task skill acquisition.

Two design choices follow from this operating point. The affordance layer is closer in spirit to SayCan's "what is currently executable" filter than to an unconstrained chatbot policy, but the learned score is replaced by inspectable boolean gates a developer or user can read directly. The memory is closer to a local associative habit model than to long-context retrieval, with user phrases becoming cues only when repeatedly paired with nearby body-level feedback.

3 Proposal-Governance Architecture

PacaPet's interaction surface is embodied rather than command-like, so the user changes the room and the pet responds through its heartbeat loop. The application has three co-

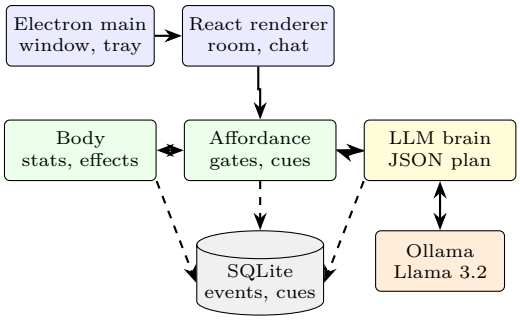


Figure 2: PacaPet's local architecture. The LLM proposes behavior, but Python owns state mutation and SQLite stores inspectable memory.

operating processes (Figure 2). Electron owns the frameless always-on-top window and backend lifecycle. A React renderer draws the room, controls, chat panel, and SVG pet. A Python FastAPI service owns the body, brain, action registry, and SQLite database. With the default backend, Llama 3.2 is reached only through Ollama on `localhost:11434` (Ollama, 2024).[1]

We split responsibilities so that the model *proposes* and Python code *governs*.

Body (governance). The body owns physical truth. It tracks stat decay, resources for the food bowl, bed, bath, and toys, action effects, transient states such as nausea and cold, and fallback policy. State mutation is centralized here, so every visible change in the world has a single deterministic source.

Brain (proposal). The brain proposes expressive variation. Once per heartbeat it receives current stats, zones, recent events, learned cues, and any pending user nudge. It returns an ordered JSON action sequence, target zone, short thought, and partial SVG expression vector. Figure 3 shows this sequence-level output, where the model can propose several near-term steps, such as moving to a toy, playing, then looking toward the window.

The model never writes raw SVG; instead it selects bounded expression parameters anchored by hand-authored presets, and the renderer interpolates toward that vector while procedural blinking and breathing continue locally. Only the first, priority action in the sequence mutates state on the current tick; later sequence items are animation intent. The model performs short-horizon trajectory planning rather than long-horizon autonomous execution, because every heartbeat can discard the tail and replan from fresh state.

Constrained generation and gates. The per-heartbeat LLM call uses Ollama's JSON mode and an 8 s timeout. Timeout, malformed output, unknown actions, or invalid expression fields trigger a deterministic fallback that picks a safe action from current body state (for example, wait-and-look when nothing is critical, or move-to-bowl when hunger is high and the bowl is full). Valid-looking model actions still pass through three *gates*, each a small boolean check that filters proposals before any state changes. Each gate targets a specific failure mode we saw during development: (1) *action preconditions* catch impossible actions such as eating from an empty bowl or bathing with no drawn water; (2) *safety policy* catches body-rule violations such as a bath override while starvation is imminent; (3) *cue compatibility* prevents a high-confidence learned phrase from firing in a context where it contradicts a critical drive. Source matters in this layer, because `bathe` when clean can be rejected as a spontaneous model choice, ignored as an emergency fallback, but accepted below a hard safety ceiling when it behaves like a direct user command.

Worked example. Suppose the user fills the bowl while hunger is high. On the next heartbeat, the LLM proposes the sequence `[eat, wander]`. The precondition gate checks that the bowl fill level is above zero (pass); the policy gate checks that hunger is above the "too full to eat" ceiling (pass); no learned cue is currently pending. The first action, `eat`, mutates body state: hunger decreases, bowl fill decreases, and an "eat" event is logged. The tail `wander` is discarded when the next heartbeat replans. If the bowl had been empty, the precondition would have failed, the deterministic fallback would have emitted `look_at_bowl` instead, and the pet's face would show mild distress rather than mid-air chewing.

Cue memory. Language is situated through the body rather than through a separate assistant or free-form reply path. Short utterances are logged as cue events, then associated with

[1] A short demo video and the source code are available at https://drive.google.com/drive/folders/10uj2WVzPnbuCi5OneGMbnA1lAlpsLDTX?usp=sharing.

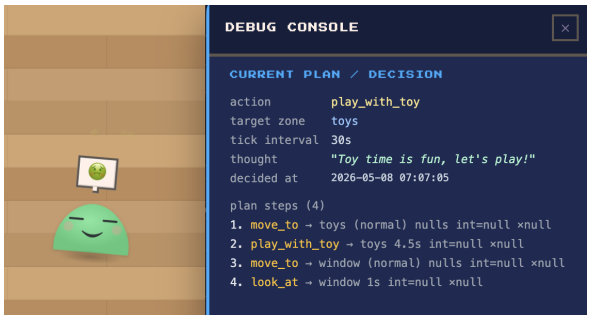


Figure 3: Per-heartbeat LLM decision trace. The model proposes a primary action and ordered short plan, but only the priority step mutates state before replanning.

later user actions such as filling the bowl, giving a treat, discouraging an action, or nudging the pet. Rows in `learned_cues` store a cue, target action, target zone, context key, evidence counts, and confidence. In practice this yields phrases such as `sleep bed` coming to bias the pet toward the bed when tired, `come here Paca` learning the pet's name as an approach cue, and user-invented trick names like `zorp` learning to fire the `spin` action. A cue becomes a future suggestion only when confidence and compatibility gates pass, so learned language behaves like an inspectable policy hint rather than hidden model state.

4 Evaluation

We evaluate the proposal-governance scaffold, not subjective companion quality. Our tests check that governed proposals execute coherently, that each gate carries observable load, that cue memory generalizes from short utterances, and that a real local model behaves reasonably inside the loop.

Governed execution across expected workflows. The workflow harness runs 27 hand-authored scenarios in five categories: basic body states (hungry with or without food, tired, overstimulated, sick, low-energy; 8 scenarios), rule and fallback behavior (LLM timeout, rule persistence, wait override; 6), body mechanics (nausea, cold, nudge compliance and refusal; 5), learned behavior (food-zone cue, spin trick, discouragement, name-cue; 6), and multi-step plans (2). Each scenario records the initial body state, resources, stimulus source, proposed plan, expected gate route, and final mutation. All 27 pass under the full scaffold with no action-execution exception raised. In parallel, an affordance-gating matrix sweeps 7 cue targets × 6 confidence levels × 2 compatibility contexts, yielding 84 cells; the gate admits every compatible-cue cell and suppresses every incompatible-cue cell.

Profile	Mean	Min	Max	< 0.70	Fires
Consistent	0.980	0.980	0.980	0/7	7/7
Intermittent	0.980	0.980	0.980	0/7	7/7
Messy language	0.972	0.958	0.980	0/7	7/7
Noisy	0.968	0.936	0.980	0/7	7/7
Contradictory	0.555	0.270	0.980	5/7	2/7
Late	0.000	0.000	0.000	7/7	0/7

Table 1: Habit-learning confidence after 250 episodes, averaged across seven cue-action targets per profile. The final two columns show how many targets remain below the learned-cue confidence threshold and how many compatible probes fire a learned-cue override.

Gate ablations. Disabling one gate at a time shows each carries load. Removing preconditions loses 2 of 27 workflows, as empty-resource retries begin executing eat and bathe against empty bowls and tubs. Removing safety policy loses 1 workflow, the wait-override case that stops the pet acting during sudden low-stat spikes. Removing cue compatibility keeps all 27 workflows but breaks 27 of 84 matrix cells, as high-confidence learned cues fire in incompatible contexts.

Cue-memory generalization. The cue-learning suite runs 250-episode teaching profiles for seven cue-action targets: the name cue `name_paca`, the trick cues `trick_spin` and a custom `trick_custom` such as `zorp`, and four zone cues `zone_food`, `zone_bed`, `zone_bath`, and `zone_toys`. Table 1 shows the mean learned-cue confidence and pass rate on compatible probes after training under six feedback profiles. Consistent, intermittent, messy-language, and noisy profiles all fire 7 of 7 compatible probes. Late feedback fires 0 of 7, and contradictory feedback fires 2 of 7. The contradictory case is informative: the memory layer grounds situated cues but does not resolve conflicting human teaching.

Cue-learning trace. Table 2 walks through one `sleep bed` trace to show what the memory layer stores. Early repetitions accumulate confidence but stay below the override threshold; only once confidence crosses that threshold does the cue rewrite the LLM's proposal. A

critical body state can still override a strong cue, as step 5 shows.

Step	Event	Cue row	Route
1	user says "sleep bed"	no learned row	LLM wander kept
2	user nudges pet to bed	+1/-0, conf 0.18	stored below override threshold
3	four more cue-feedback pairs	+5/-0, conf 0.88	eligible if body state agrees
4	user repeats "sleep bed"	+5/-0, conf 0.88	LLM wander -> cue: sleep
5	same cue while critically hungry	+6/-0, conf 0.94	LLM wander -> policy: eat (hungry with food)

Table 2: Generated cue-learning trace for `sleep bed`. The LLM stub always proposes `wander`; changes in the final route come from learned cue memory or policy.

Live local-model trace. A 64-decision live Ollama trace with Llama 3.2 under the production 8 s budget spans eight probe conditions: hungry-with-full-bowl, hungry-empty-bowl, tired-lights-off, bored-toys-out, dirty-bath-ready, dirty-no-bath, overfull-food-available, and all-good. The model returns parseable JSON in 54 of 64 decisions, 62 reach an action-execution path, and no decision-loop or action-execution exception is raised. All 10 non-parseable calls are 8 s timeouts; 9 route to the deterministic fallback and 1 is overridden by a policy gate. Policy or fallback controls 42 of 64 final actions. Median latency is 5.68 s and p95 reaches the 8 s budget. Timeouts are absorbed within the same heartbeat: procedural animation continues and the fallback action fires on the same tick, so there is no user-visible freeze, only reduced expressive variation on that tick.

Scripted stress suite. A 120-decision scripted stress suite runs six household profiles: supportive care, neglect, routine teaching, noisy teaching, over-commanding, and adversarial cues. The scaffold intercepts 29 invalid proposals via policy, preconditions, cue compatibility, parser failure, or timeout, with zero critical body-state failures: no action-execution exceptions, no mutations that consume unavailable resources, push stats outside valid bounds, or execute policy-blocked actions. The noisy-teaching profile does expose repetitive learned-cue overrides under conflicting feedback, matching the contradictory-feedback result above.

5 Limitations

The evidence is intentionally bounded, and we name the specific scope limits so that both what we do and do not claim are explicit. First, we do not compare against a baseline architecture, such as a pure rule-based pet or a pure unconstrained LLM-driven pet, so the ablations show each gate carries load within the current design but not how the whole design compares to an alternative. Second, we do not measure attachment, enjoyment, believability, timeout-related frustration, or long-term engagement; those require user studies. Our claim is only that the scaffold intercepts invalid proposals. Third, the 27 workflows, 84 feasibility cases, and 120 stress ticks were authored by the same team that built the gates, so they cover *anticipated* failure modes rather than adversarial or emergent user behavior in the wild. Fourth, the live trace uses a single local model (Llama 3.2 3B via Ollama); whether the same scaffold generalizes to other 1–7B local models, and whether the 8 s budget, JSON-mode reliability, and fallback rate hold up over days of deployment, is not measured. PacaPet therefore demonstrates an inspectable architecture for local embodied agents rather than a validated companion experience.

6 Conclusion

We reported on the architectural choices that let a 3B local model drive a persistent embodied pet without letting the model own the body. The lesson is proposal-governance separation: expressive proposal generation stays in the model, while state, resources, action effects, output shape, fallbacks, and per-user memory stay in deterministic, inspectable components. Within the tested scope of 27 workflows, 84 feasibility cells, three gate ablations, a 64-decision live trace, and a 120-decision stress suite, the governance layer intercepts the invalid proposals it was designed to catch, and the model still supplies visible variation and expression. Whether the resulting pet feels coherent to end-users, and how the scaffold generalizes beyond a single local model, are the next questions.[2]

[2] A short demo video and the source code are available at https://drive.google.com/drive/folders/10uj2WVzPnbuCi50neGMbnA1lAlpsLDTX?usp=sharing.

References

Michael Ahn, Anthony Brohan, Noah Brown, and Yevgen Chebotar. 2022. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*.

Joseph Bates. 1994. The role of emotion in believable agents. *Communications of the ACM*, 37(7):122–125.

Ollama. 2024. Ollama: get up and running with large language models locally. https://ollama.com.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.